CiboAmico

Technical Documentation

Project Report – Ingegneria del Software (ISPW)
Università degli Studi di Roma "Tor Vergata"
A.A. 2025/2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Goal

CiboAmico is a desktop application that supports a local food marketplace: it lets a household monitor its pantry (*frigo/dispensa*), suggest compatible recipes based on what is actually available, and order fresh products directly from local farmers and small suppliers. The system bridges daily food management with short food-supply chains, reducing waste and promoting local production.

The project is developed as the final assignment of the *Ingegneria del Software* (ISPW) course at Università di Roma "Tor Vergata", under the supervision of Prof. Falessi and Prof. De Angelis (A.A. 2025/2026).

1.2 Scope

The system covers four functional areas:

- **Pantry management**: add, list (ordered by expiry date) and monitor products in the household inventory;
- **Recipe engine**: find recipes compatible with the available ingredients, including an innovative ingredient-substitution mechanism;
- **Shopping list**: compute the missing ingredients for a chosen recipe;
- **Marketplace**: publish products as an approved local vendor, place single direct orders, track order state, and approve vendors and recipes as an administrator.

1.3 Actors

The system defines four primary actors:

- **Client**: manages the pantry and uses the recipe engine and the marketplace to buy products;
- **Vendor**: a local farmer or supplier who, once approved by the administrator, publishes products with prices;

- **Nutritionist**: the only actor allowed to create recipes, which are published with a *PROPOSED* state;
- **Administrator**: approves vendors and recipes.

Two external systems are used through adapters: **OpenFoodFacts** (barcode-based product lookup) and the e-mail notification service (*Jakarta Mail*).

1.4 Hardware and Software Requirements

CiboAmico is a desktop application with the following minimum runtime requirements:

- Java Runtime Environment (JRE) 17 or later, with JavaFX 17+;
- 4 GB of RAM and 500 MB of free disk space;
- optional MySQL 8 server (the JDBC persistence mode); the file-system (JSON) and in-memory demo modes require no external services (NFR-01);
- a working e-mail account for the notification service (Jakarta Mail adapter) and an Internet connection for the OpenFoodFacts barcode lookup.

1.5 Implementation Remarks

The project deliberately implements the use cases under evaluation (UC-01..UC-08, UC-11); UC-10 is documented for model completeness but is excluded from the implementation and evaluation scope (see Chapter 5).

1.6 Document Structure

The remainder of this document is organised as follows: Chapter 2 discusses related systems and the innovation elements of CiboAmico; Chapter 3 lists functional and non-functional requirements; Chapter 4 presents the user stories (INVEST); Chapter 5 details the use cases with their internal steps; Chapter 6 shows the interface storyboards; Chapter 7 describes the design (BCE architecture, VOPC, activity, sequence and state diagrams); Chapter 8 presents the test plan and results; Chapter 9 covers exception handling and persistence; finally, Chapter 10 collects the public links (repository, SonarCloud report, demonstration video) and the conclusions.

Chapter 2

Related Systems

Several existing applications address parts of the problem space covered by CiboAmico. Table 2.1 compares them along the dimensions that are central to this project.

Table 2.1: Comparison with related systems.

System	Strengths	Limitations
Too Good To Go	Reduces food waste; large local network; simple ordering flow.	No pantry/inventory management; no recipe suggestions; no vendor publication control.
MyFoodRepo	Recipe database with ingredient filtering.	No local marketplace; no direct order from farmers; no substitution engine.
Supermarket apps (loyalty)	Shopping lists and discounts.	No integration with home inventory; no barcode-to-pantry flow.

2.1 Innovation Elements

CiboAmico introduces three distinguishing elements with respect to the systems above (design decisions D-01/D-02/D-03):

1. **Ingredient substitution engine (D-01)**: when an ingredient is missing, the recipe engine proposes equivalent substitutes with a conversion factor (e.g. 0.5 kg of tomato purée for 1 kg of fresh tomatoes), implemented as a *Strategy* pattern;
2. **Dynamic role metamorfosi (D-02)**: a user can acquire the vendor or nutritionist role at runtime (Whole-Part pattern) without restructuring the class model;
3. **Single direct order (D-03)**: the marketplace adopts a one-buyer–one-vendor direct order with no cart, tailored to the short-supply-chain scenario.

2.2 Design Rationale

This section records the defence arguments for the three innovation elements, as they are the most frequently questioned during the oral examination.

D-01 (Ingredient substitution engine) The substitution engine implements the *Strategy* pattern: each matching policy (`StrictMatchingStrategy`, `SubstitutionMatchingStrategy`) is an interchangeable algorithm, and new substitution policies (by price, by calories, by proximity) can be added without modifying the existing code (Open/Closed Principle). The engine is exposed through a single entry point, the `RicettaMatchingFacade` (Facade pattern), which decouples the presentation layer from the matching policies.

D-02 (Dynamic role metamorfosi) The classical alternative to role dynamism is rigid inheritance, which would force the system to destroy the in-memory instance and create a new one (with different references) every time a `Client` acquires the `Vendor` or `Nutritionist` role. CiboAmico adopts the *Whole-Part* pattern: `Utente` (Whole) aggregates a collection of roles (Part), so the role metamorphosis happens at runtime without altering the identity of the object in memory. This avoids the Fragile Base Class problem and keeps the model extensible.

D-03 (Single direct order) The marketplace deliberately avoids the cart idiom: the short-supply-chain scenario is one buyer–one vendor per transaction. The order is created directly in the *CREATED* state and progresses through the state machine defined by BR-04; the vendor is notified asynchronously through the *Observer* pattern (`VenditoreNotifier`).

Chapter 3

Requirements

This chapter lists the functional and non-functional requirements of CiboAmico. Every requirement is expressed with an active verb, is independent from the graphical interface and is objectively testable.

3.1 Functional Requirements

3.1.1 **FR-01:** The system shall add a product to the household inventory with name, quantity, expiry date, storage position and unit of measure.

3.1.2 **FR-02:** The system shall present the inventory ordered by ascending expiry date.

3.1.3 **FR-03:** The system shall flag the products whose expiry date falls within the next three days.

3.1.4 **FR-04:** The system shall provide the details of a product associated with a scanned barcode or a specific search query.

3.1.5 **FR-05:** The system shall compute the list of ingredients missing from the inventory for a selected recipe.

3.1.6 **FR-06:** The system shall publish a product on the marketplace only if the vendor is approved and the price is greater than zero.

3.1.7 **FR-07:** The system shall create a single direct order for one buyer and one vendor, tracking the order state transitions.

3.1.8 **FR-08:** The system shall create a recipe with at least two ingredients, published in the *PROPOSED* state.

3.1.9 **FR-09:** The system shall approve or suspend a vendor.

3.1.10 **FR-10:** The system shall approve or reject a proposed recipe.

3.1.11 **FR-11:** The system shall authenticate a user by e-mail and password, managing a session.

3.1.12 **FR-12:** The system shall find the recipes compatible with the available inventory, including equivalent ingredient substitutions.

3.1.13 **FR-13:** The system shall present to a vendor the orders received for their products, with their current state, and shall update the state upon the vendor's action.

3.2 Business Rules

Business rules constrain the functional behaviour:

3.2.1 **BR-01:** A product whose expiry date is in the past cannot be sold.

3.2.2 **BR-02:** Only vendors in the *APPROVED* state can publish products.

3.2.3 **BR-03:** The available quantity of a product cannot be negative.

3.2.4 **BR-04:** The order state machine restricts transitions exclusively to: CREATED → CONFIRMED/ANNULLED, CONFIRMED → IN_DELIVERY/ANNULLED, IN_DELIVERY → DELIVERED.

3.2.5 **BR-05:** A recipe must contain at least two ingredients.

3.2.6 **BR-06:** The price of a marketplace product must be greater than zero.

3.3 Non-Functional Requirements

3.3.1 **NFR-01:** The system shall support at least three persistence modes (relational database, local JSON files, in-memory demo), selectable at application startup.

3.3.2 **NFR-02:** The system shall protect data access against SQL-injection by using prepared statements.

3.3.3 **NFR-03:** The system shall store user passwords using a one-way cryptographic hash with salt.

3.3.4 **NFR-04:** The line coverage of the automated test suite shall be at least 80% on the application logic.

3.4 Requirements–Use-Case Traceability

Table 3.1 maps every functional requirement to the use cases that realise it, guaranteeing full traceability from requirements to code.

Table 3.1: FR to UC traceability.

FR	UC	FR	UC	FR	UC
FR-01	UC-01	FR-05	UC-03	FR-09	UC-08
FR-02	UC-01	FR-06	UC-05	FR-10	UC-08
FR-03	UC-01	FR-07	UC-04	FR-11	UC-11
FR-04	UC-01	FR-08	UC-07	FR-12	UC-02
				FR-13	UC-06

Chapter 4

User Stories

Each user story follows the mandatory format *"As a <role>, I want to <capability>, So that <benefit>"* and satisfies the INVEST criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable). The login is not modelled as a story: per the course rule, authentication is a precondition of the primary use cases and is realised by UC-11 through the *include* relationships.

Table 4.1: User stories (INVEST).

ID	User story
US-01	As a client, I want to add a product to my pantry with its expiry date, So that I can always know what food I have and when it expires.
US-02	As a client, I want to view my pantry ordered by expiry date, So that I consume the most urgent products first.
US-03	As a client, I want to receive a warning for products expiring within three days, So that I avoid food waste.
US-04	As a client, I want to scan or search a product barcode, So that I add its details without manual typing.
US-05	As a client, I want to know which ingredients are missing for a recipe, So that I build an exact shopping list.
US-06	As a client, I want to order a product directly from a local vendor, So that I receive fresh food from my territory.
US-07	As a client, I want to track the state of my orders, So that I know when the delivery will arrive.
US-08	As a vendor, I want to publish my products with a price once I am approved, So that local customers can buy them.
US-09	As a nutritionist, I want to create recipes with at least two ingredients, So that they are proposed to clients after approval.
US-10	As a client, I want to view my order history, So that I can review my past purchases. *(excluded from evaluation, see UC-10)*
US-11	As an administrator, I want to approve vendors and recipes, So that only trusted actors and quality content reach the marketplace.

Chapter 5

Use Cases

This chapter presents the use-case model of CiboAmico. The overview diagram 5.1 shows the system boundary, the four primary actors, and the external actors *OpenFoodFacts* and *Payment Service*. Each use case is described with preconditions, postconditions, main success scenario (MSS) and extensions. All steps are written in the simple present with active verbs; no GUI detail is included. Use case **UC-10** (grey in the diagram) is excluded from the evaluation scope: it documents a domain capability that is not implemented, keeping the numbering continuous and the model coherent with the code.

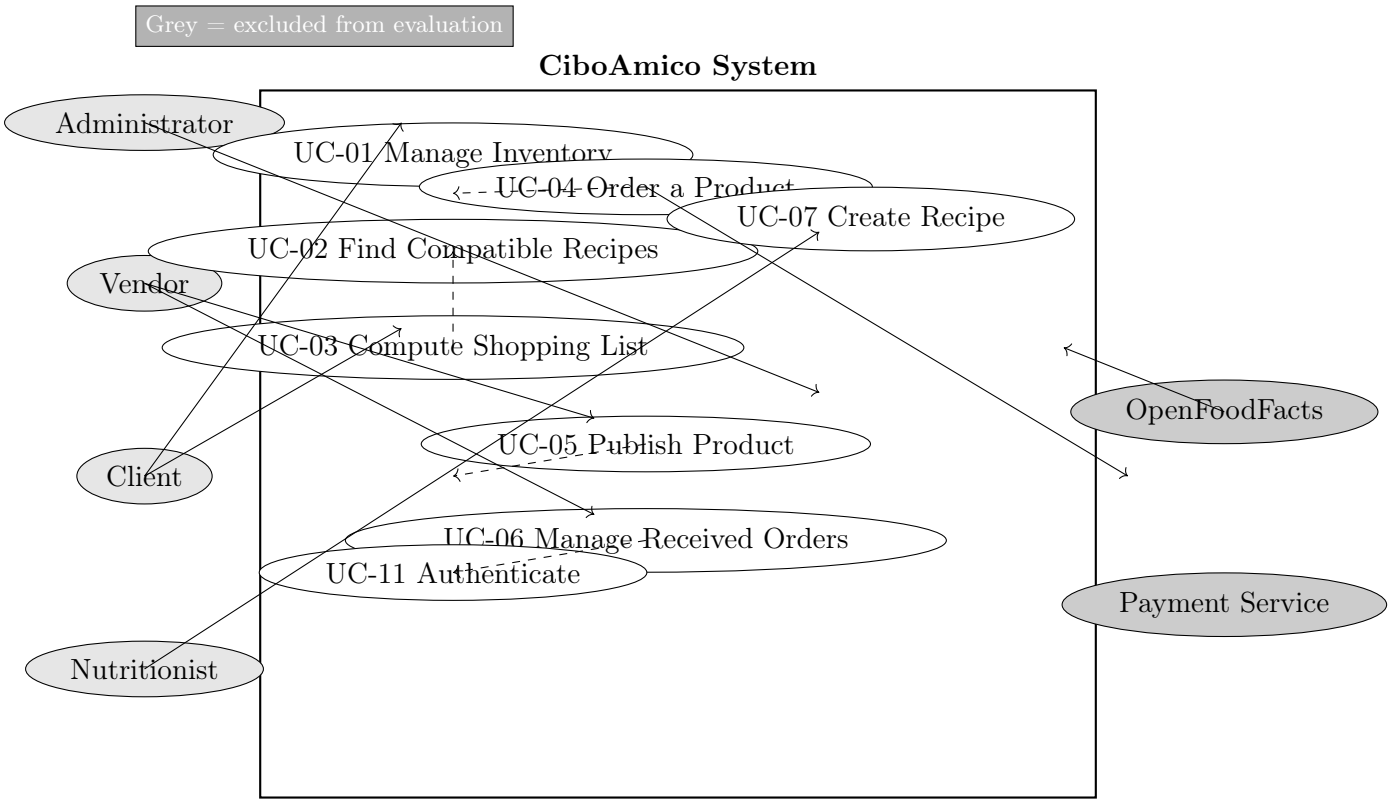

Figure 5.1: Use-case diagram (grey UC-10 excluded from evaluation).

5.1 UC-04 Order a Product (exemplary template)

Actors: Client (primary), Payment Service (external).

Preconditions: the Client is authenticated; the product is published on the marketplace; the inventory of the vendor contains the requested quantity.

Postconditions: an order in the *CREATED* state is recorded for the buyer and the vendor; the vendor receives a notification.

Main Success Scenario (MSS):

1. The Client selects a product from the marketplace.
2. The System verifies the availability of the product.
3. The System shows the order summary with the total price.
4. The Client confirms the order.
5. The System records the order with state *CREATED*.
6. The System notifies the vendor of the new order.

Extensions:

- **2a. Product out of stock:**
    1. The System notifies the Client of the product unavailability.
    2. The flow returns to step 1.
- **5a. Invalid state transition (BR-04):**
    1. The System rejects the transition and keeps the previous state.

5.2 Other Use Cases (summary)

UC-01 Manage Inventory

Actors: Client. **Pre:** authenticated. **MSS:** (1) the Client provides product data; (2) the System validates the data (FR-01); (3) the System stores the product in the inventory; (4) the System confirms. **Extensions:** 2a missing mandatory data; 2b invalid quantity. **Post:** product stored and visible in the expiry- ordered list (FR-02).

UC-02 Find Compatible Recipes

Actors: Client. **Pre:** inventory not empty. **MSS:** (1) the System retrieves the inventory; (2) the System retrieves the approved recipes; (3) the System matches ingredients (strict or with substitutions, D-01); (4) the System presents the compatible recipes. **Post:** the client sees only the feasible recipes.

UC-03 Compute Shopping List

Actors: Client. **Pre:** a recipe exists and is approved. **MSS:** (1) the Client selects a recipe; (2) the System compares the recipe ingredients with the inventory; (3) the System presents the missing ingredients with quantities. **Post:** an exact shopping list is shown (FR-05).

UC-05 Publish Product

Actors: Vendor. **Pre:** vendor approved (BR-02). **MSS:** (1) the Vendor provides product data with price; (2) the System validates price and expiry (BR-06, BR-01); (3) the System publishes the product. **Extensions:** 2a price ≤ 0; 2b expired product.

UC-06 Manage Received Orders

Actors: Vendor. **Pre:** at least one order exists for the vendor. **MSS:** (1) the System presents the orders in *CREATED* or *CONFIRMED* state; (2) the Vendor updates the state of an order; (3) the System validates the transition (BR-04); (4) the System stores the new state and notifies the buyer. **Extensions:** 3a invalid transition.

UC-07 Create Recipe

Actors: Nutritionist (the only role allowed). **Pre:** authenticated as nutritionist. **MSS:** (1) the Nutritionist provides the recipe with at least two ingredients (BR-05); (2) the System creates the recipe in *PROPOSED* state; (3) the System stores the recipe. **Extensions:** 1a fewer than two ingredients.

UC-08 Manage Pending Approvals

extbfActors: Administrator. extbfPre: administrator authenticated. extbfMSS: (1) the Administrator requests the pending approvals; (2) the System retrieves the categorised list of pending vendor registrations and recipes in *PROPOSED* state; (3) the Administrator selects a pending item; (4) the System presents its details; (5) the Administrator approves, suspends or rejects the item; (6) the System updates the state, notifies the interested actor and confirms. extbfExtensions: 2a empty queue; 5a rejection/suspension returns to step 2. extbfPost: item status updated (APPROVED/SUSPENDED for vendors, APPROVED/REJECTED for recipes).

UC-11 Authenticate

Actors: any primary actor. **MSS:** (1) the Actor provides e-mail and password; (2) the System validates the credentials (NFR-03); (3) the System opens a session. **Extensions:** 2a invalid credentials. **Post:** the session is active; *include* from UC-01..UC-07.

Chapter 6

Storyboards

The interface follows an intentionally *aseptic and minimal* style: the visual appearance is not part of the evaluation, while the interaction logic must exchange only beans (DTO) with the controllers. Each screen is available in two themes: *colour* and *black-and-white* (high contrast) for accessibility.

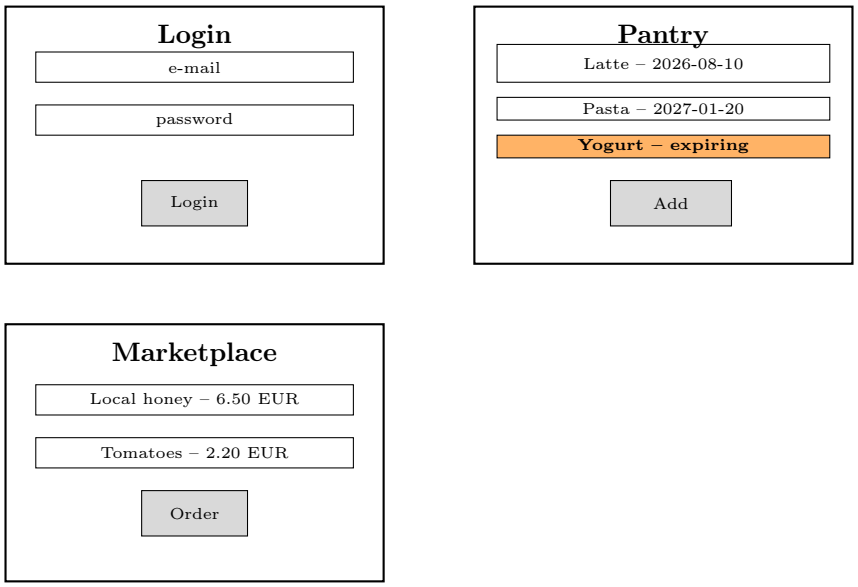

Figure 6.1: Minimal storyboards (colour theme): login, pantry with expiry warning, marketplace.

6.1 State of the system

Every screen makes the system state explicit: the current view, the clickable actions, and the output area. The navigation is linear: *Login* → *Home* → *Pantry / Recipes / Marketplace*, with role-based entry points (vendor, nutritionist, administrator).

Chapter 7

Design

7.1 BCE Architecture (VOPC)

CiboAmico follows the **Boundary–Control–Entity** pattern. The VOPC (view of participating classes) respects the following rules:

- each use case owns exactly **one application controller**;
- boundaries exchange data with controllers **exclusively via beans (DTO)**: no entity reaches the view;
- controllers are **stateless** and resolve the current user from the session singleton;
- business logic resides in the entities.

The final model is summarised in Table 7.1.

Table 7.1: BCE classes (1 controller per use case).

UC	Boundary	Control	Main Entities
UC-01	InventoryView	GestisciInventarioController	ProdottoInventario
UC-02	RecipesView	TrovaRicetteController	Ricetta, ProdottoInventario
UC-03	ShoppingListView	GestisciListaSpesaController	Ricetta, Ingrediente
UC-04	MarketplaceView	OrdinaProdottoController	Ordine, Prodotto
UC-05	CatalogView	GestisciCatalogoVenditoreController	Prodotto, RuoloVenditore
UC-06	OrdersView	GestisciOrdiniRicevutiController	Ordine
UC-07	RecipeEditorView	GestisciRicetteNutrizionistaController	Ricetta, RuoloNutrizionista
UC-08	AdminView	ApprovazioneController	RuoloVenditore, Ricetta
UC-11	LoginView	AutenticazioneController	Utente

7.2 GoF Patterns

- **Singleton** – `SessionManager`, `ApplicationModeManager`: single session and application mode;

- **Abstract Factory** – `DAOFactory` with `DemoDAOFactory`, `FSDAOFactory`, `JDBCDAOFactory`: coherent family of DAOs switchable at startup (NFR-01);
- **Strategy** – `StrictMatchingStrategy`, `SubstitutionMatchingStrategy`: interchangeable matching algorithms;
- **Facade** – `RicettaMatchingFacade`: single entry point to the recipe engine;
- **Observer** – `OrdineSubject`, `UtenteNotifier`, `VenditoreNotifier`: order-state notifications;
- **Adapter** – `OpenFoodFactsAdapter`, `JakartaMailAdapter`: isolates external systems;
- **Whole-Part** – `Utente` aggregates `Ruolo` (dynamic role metamorfosi, D-02).

7.3 Activity Diagram (UC-04)

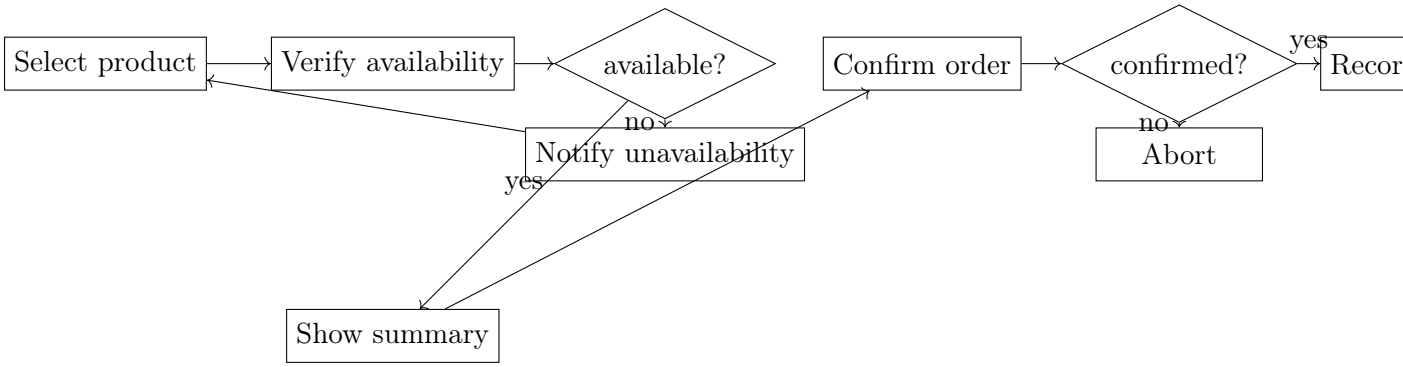

Figure 7.1: Activity diagram of UC-04 (simplified).

7.4 Sequence Diagram (UC-04)

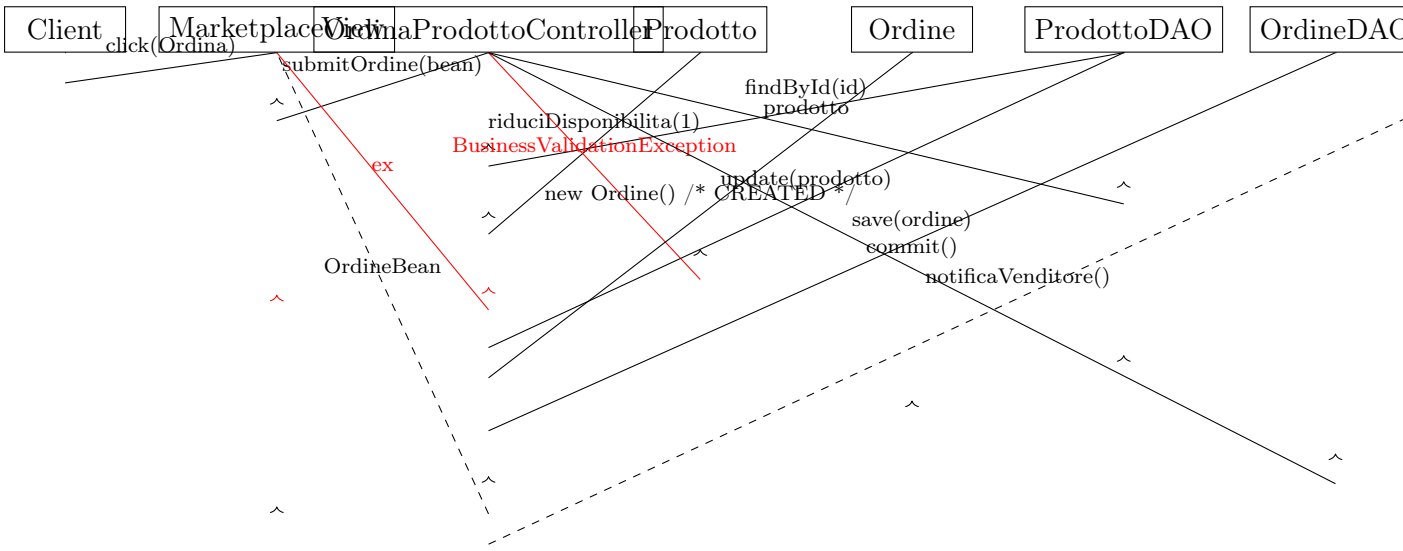

Figure 7.2: Sequence diagram of UC-04: synchronous calls (full arrows), return bean (dashed), exception propagation (red), async Observer notification (open arrow).

7.5 State Diagram (Ordine, BR-04)

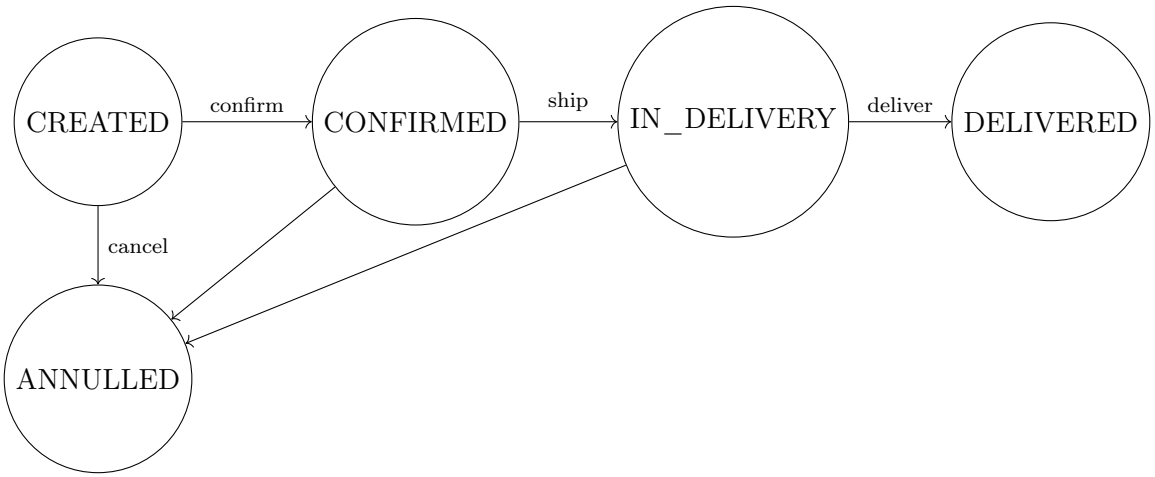

Figure 7.3: State diagram of Ordine (trigger / behaviour, guards mutually exclusive per BR-04).

Chapter 8

Testing

8.1 Strategy

The test suite is based on **JUnit 5** and **Mockito**. The strategy follows a bottom-up approach: entities are tested first (business rules), then controllers (use-case flows), then the persistence layer (demo and file system round-trips), and finally the GoF patterns (facade, strategy, observer, adapter).

- **Unit tests**: business rules on entities (BR-01..BR-06), strategy matching, observer notifications, bean DTO validation;
- **Integration tests**: controllers against the demo DAO factory, covering every use case (UC-01..UC-09, UC-11);
- **Persistence tests**: JSON file-system DAOs with serialisation round-trips and Gson local-date adapters.

The DAO JDBC implementations require a real MySQL instance and the JavaFX boundaries require a display; they are excluded from the automated coverage and covered by manual tests (documented in the repository).

8.2 Coverage

Table 8.1: Test results and coverage.

Metric	Value
Total automated tests	82
JUnit 5 + Mockito	all passing
Line coverage (JaCoCo)	82.5%
Quality gate minimum	80%
SonarCloud new-code coverage	81.3%
Bugs / vulnerabilities / code smells	0 / 0 / 0
Duplications	0%

8.3 Regression testing

Every defect found during the NotebookLM-assisted verification was fixed with a dedicated regression test. In particular, the order controller now resolves the vendor from the product's whole-part back-reference, and the purchase flow (UC-04) is covered by three targeted tests asserting state transitions (BR-04).

Chapter 9

Exception Handling and Persistence

9.1 Exception Handling

Exceptions are modelled as a *domain-specific* hierarchy that never exposes physical persistence failures to the view layer:

- `DAOException` (runtime): wraps physical failures such as `SQLException` raised by the JDBC layer, converting them into a domain-abstraction error at the DAO boundary;
- `BusinessValidationException` (runtime, base of the business branch): raised by entities and controllers for business-rule and validation violations (e.g. invalid quantity, non-approved vendor, self-purchase of one's own product);
- `InvalidStateTransitionException` (extends `BusinessValidationException`): raised when an order state transition violates the state machine (BR-04);
- `AutenticazioneException` (extends `BusinessValidationException`): raised for malformed e-mail or invalid credentials during authentication (UC-11).

This small, domain-specific hierarchy gives the presentation layer an expressive oracle: each view catches exactly the domain error it needs (e.g. a credential failure), while technical persistence failures are wrapped by `DAOException` and never leak to the view. The automated tests assert the most specific exception type, avoiding weak oracles.

9.2 Persistence

The system supports **three persistence modes** (NFR-01), selected at startup through the `ApplicationModeManager` and the `DAOFactory` (Abstract Factory):

Table 9.1: Persistence modes.

Mode	Technology	Use
JDBC	MySQL (prepared statements, NFR-02; transactional saves)	production
File system	JSON via Gson (local-date adapters)	portable persistence
Demo	in-memory maps (shared factory instance)	tests and demo

Every DAO (utente, prodotto, ricetta, ordine) is defined by an interface and implemented three times; the `DAOFactory` guarantees that the whole family is coherent and selected at runtime. The JDBC implementations use `PreparedStatement` (NFR-02) and wrap each write in an explicit transaction (auto-commit off, `commit`/`rollback`), improving robustness. Password storage uses a one-way SHA-256 hash with a fixed salt (NFR-03). The physical schema of the database is intentionally minimal: the evaluation focuses on the interaction between controllers, entities and the DAO layer.

Chapter 10

Public Links and Conclusions

10.1 Public Links

- **Source repository (GitHub):** https://github.com/Damiano-o/ciboamico – public repository with a logical commit history (project setup, domain model, persistence, GoF patterns, controllers, boundaries, tests) and a feature-branch flow for the ongoing work.
- **Static analysis (SonarCloud):** https://sonarcloud.io/project/overview?id=Damiano-o_ciboamico – quality gate **passed**: 0 bugs, 0 vulnerabilities, 0 code smells, 0% duplications, coverage above 80%; graphical components are excluded from the metrics (documented in the build configuration).
- **Demonstration video:** `.mp4` – end-to-end walkthrough of the application (see the repository releases).

10.2 Conclusions

CiboAmico applies the software-engineering practices studied in the course: requirements elicitation and analysis (functional requirements, user stories INVEST, use cases with internal steps), design with the BCE pattern and seven GoF patterns, a test-first quality culture (82 automated tests, 82.5% coverage), static-analysis quality gate, and a clean git workflow on a public repository.

The main innovation elements are the ingredient-substitution recipe engine (Strategy), the dynamic role metamorfosi (Whole-Part) and the single direct order for the short food-supply chain.

10.3 Future Work

- complete the remaining minimal JavaFX views (colour and black-and-white themes);
- implement the concrete e-mail adapter against a real SMTP server;
- implement UC-10 (order history) to complete the client experience;
- continuous integration pipeline on GitHub Actions with SonarCloud analysis on every push.